商家在 gomall 里是怎么活的

BossID 字段在、商家自助 API 缺：从” 商家用 admin 后台” 到” 商家用商家后台”

gomall · 业务侧 deck

今日大纲

业务困局：商家在 gomall 里没有” 自己的后台”

商家用户旅程：哪一站有路、哪一站没路

三层鉴权边界：把 router 翻译成商家视角

数据基础：BossID 字段在哪

商家后台 MVP：第一版接口设计

商家看板：信息架构四象限

商家告警的业务规则

商家结算与提现：资金路径

admin vs merchant：边界由谁守

压测数据告诉我们什么

遗留风险与路线图

业务困局：商家在 gomall 里没有” 自己的后台”

一句话：BossID 字段早就在了，商家接口一个都没有

- gomall 五张核心表里，**BossID 字段都已经埋好**：order、product、cart、favorite、skill_product。
- 但全站没有任何一个 `/api/v1/merchant/*` 路由——商家想看自己店铺数据**进不去**。
- 现状导致三件事：
  - 商家想看”我今天卖了多少” → 没接口，要么找客服查、要么直接看 admin 后台（越权）。
  - 商家想知道”哪个 SKU 快没库存” → 没告警，等用户下不了单才发现。
  - 商家上架商品调 `product/create` ——这接口在 authed group 下，**任何登录用户都能上架**。

routes/router.go:60–62 product/create 三件套都在 authed 下，没 RequireRole；:135–141 admin sub-group 只有 3 个 handler

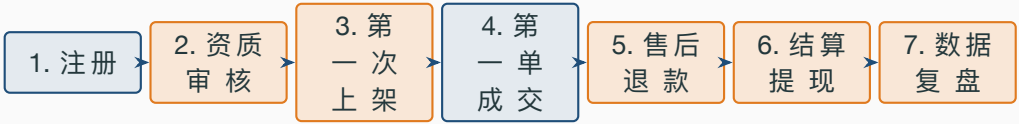
现状对业务的真实伤害

- **招商困难**：BD 谈一个新商家，洽谈阶段只能开 admin 账号给他——商家立刻看到**所有店铺**数据，谈判桌上当场翻车。
- **客服压力**：商家把客服当后台用——” 帮我查下我店铺今日订单”、” 帮我看下 SKU 12345 的库存”。客服 1 个人对 N 个商家，挤兑出的工单全是低价值查询。
- **权限边界没法解释**：`product/delete` 已经有 `WHERE id=? AND boss_id=?`，但**没任何中间件保证” 调用者一定是 boss_id 这个人”** ——数据查询正确，权限模型却没成形。

repository/db/dao/product.go:67–72 DeleteProduct 已带 boss_id 隔离；但 router.go:62 没有 RequireMerchant

商家用户旅程：哪一站有路、哪一站没路

商家七步旅程 + gomall 现状



站	业务诉求	gomall 现状	命中表/接口
1	注册	✓复用 user/register	user 表 Role='user'
2	资质审核	缺	无 merchant_profile 表
3	上架商品	部分	product/create 任谁都能调
4	第一单成交	✓但商家看不到	order 表已有 boss_id
5	售后退款	缺	无商家侧退款审批
6	结算提现	缺	无 settlement / withdraw
7	数据复盘	缺	无销量 / 转化看板

"部分"的代价： `product/create`

无角色校验，任何注册用户调一次自己就成了"店主"。gomall 业务定位是 B2C 自营 + 第三方品牌**不是闲鱼**；上架前必须资质审核。*repository/db/model/order.go:12 BossID；user.go:23 Role 只有 user/admin 两态；router.go:60 product/create*

三层鉴权边界：把 router 翻译成商家视角

三层墙：商家应该站在中间那层

角色	能看	能改	路径	现状
C 端用户	自己的订单	自己资料	`/api/v1/*`	✓
商家	自店订单/商品/库存	自店范围	`/api/v1/merchant/*`	**缺**
admin	全部	全部	`/api/v1/admin/*`	部分

三层各自的最坏情况：C 端越权 → 看到别人订单（泄露电话/地址）；商家越权 → 看到其他商家销售（竞业/抄品）；admin 越权 → 一切（必须审计，见 deck 14）。*routes/router.go:46 authed*；*:135–141 admin*；*middleware/rbac.go:48–86 RequireRole*；*model/user.go:30–31 Role* 常量

数据基础：BossID 字段在哪

BossID 散落在 5 张表 + 现有 dao 用法

位置	字段或语句	业务含义
model/order.go:12	BossID uint	这单卖家是谁
model/product.go:23	BossID uint	这商品归谁
model/cart.go:10	BossID uint	加购的店铺归属
model/favorite.go:11	BossID uint	收藏背后的店主
model/skill_product.go:6	BossId uint	秒杀活动发起商家
dao/product.go:69	DELETE WHERE id AND boss_id	商家删自己商品
dao/cart.go:61	SELECT WHERE user/product/boss	加购” 店内唯一性”
dao/order.go:135	o.boss_id AS boss_id (JOIN)	订单详情带卖家
dao/favories.go:33	JOIN user ON u.id=f.boss_id	关注的店铺

所有 boss_id 用法

都是”C 端用户视角” ——拿来展示” 这商品/订单/购物车的卖家是谁”，**从来没有” 以 boss_id 为主语” 的查询**。

> 有字段 ≠ 有能力。商家自助接口的本质是把 boss_id **从外键升级为查询主体**。

商家后台 MVP：第一版接口设计

接口设计草案：沿用 admin group 写法

Listing 1: 对照 *routes/router.go:135–141 admin* 子组，新增 *merchant* 子组

```
// 商家子组：先 authed，再 RequireRole("merchant")
merchant := authed.Group("/merchant")
merchant.Use(middleware.RequireRole("merchant"))
{
   merchant.GET ("orders", MerchantListOrders) // WHERE boss_id = ctx.userID
   merchant.GET ("orders/stats", MerchantOrderStats) // 日/周/月销量
   merchant.GET ("products", MerchantListProducts)
   merchant.POST("products", MerchantCreateProduct) // 替换裸 product/create
   merchant.GET ("inventory", MerchantInventoryStatus) // 低库存告警
   merchant.POST("coupon/batch", MerchantCreateCoupon)
   merchant.GET ("settlement", MerchantSettlement) // 结算/提现
}
```

照搬 admin 写法：`authed.Group` + `Use(RequireRole)`，复用 30s 角色缓存；`RequireRole` 已支持变参，无需改实现。*middleware/rbac.go:48–86 RequireRole(allowed...string)*

MVP 七接口的业务优先级

接口	业务紧迫度	实现工作量
GET orders	高（替代客服查单）	小（已有 order dao）
GET orders/stats	高（招商对外）	中（新增聚合 SQL）
GET inventory	高（防超卖恶化）	小（复用库存表）
POST coupon/batch	中（已有，需限店）	小（加 WHERE boss_id）
GET products	中	小
POST products	中（替换裸接口）	小（迁移 + 加 RBAC）
GET settlement	低（结算还没做）	大（涉及对账 / 第三方）

业务侧建议顺

序：先做**读接口**（让商家**看得到**），再做**告警接口**（让商家**被通知到**），最后做**结算接口**（让商家**拿得到钱**）。三步对应” 信任三段论”：能看 → 能管 → 能赚。

商家看板：信息架构四象限

商家看板四象限 + SQL 草图

今日营收
GMV / 客单价

待发货
type=2 未发

库存告警
num<safety

客诉
退款率/待回

象限	查询
今日营收	`SELECT COUNT,SUM(money) FROM order WHERE boss_id=? AND DATE=CURDATE AND type=2`
待发货	`WHERE boss_id=? AND type=2 ORDER BY created_at`
库存告警	`WHERE boss_id=? AND num < safety_stock`
客诉	`aftersale_ticket WHERE boss_id=?`（待建）

已有索引：`order.user_id` 走索引（C 端用），`boss_id` 没建索引。商家维度查询前**先 ALTER TABLE order ADD INDEX idx_boss_date(boss_id, created_at)**，否则像 deck 13 那样翻全表。*repository/db/model/order.go:16 Type 1* 未支付/2 已支付；*product.go:22 Num* 即可用库存；*dao/order.go:50* 现有 *user_id* 索引参考

商家告警的业务规则

三条核心告警规则 + 阈值设计

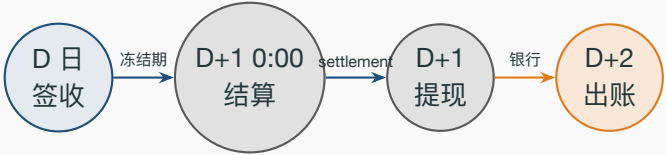
规则	触发	通道	接收方
库存阈值	num < safety_stock	outbox `inventory.low`	商家钉钉
退款异常	单日退款率 > 10%	outbox `aftersale.spike`	平台 + 商家
异常订单	单订单金额 > 均值 5×	outbox `order.suspect`	风控审查

- 三条规则共用机制：**业务事务里写 outbox** → `publisher.loop` 异步投递。同步发短信 = 网关挂一次下单全卡死。
- **阈值不是 0**：库存等于 0 告警 = 已卖断货 = 用户买不到。要用 `safety_stock`（按品类 = 7 天销量）。
- **safety_stock 谁定**：商家自填大部分填 0；平台算准但要 7 天历史。**折中**：上架前 7 天按品类均值兜底，7 天后切实际销量。
- **频率收敛**：同 SKU 24h 只发一次，防商家被噪音淹没。

service/outbox/publisher.go:65–86 drainBatch；model/product.go:22 Num；docs/slides/04-outbox-saga.tex 配套

商家结算与提现：资金路径

T+1 结算 + 隔离账户



账户	含义	状态
冻结户 A	已支付未发货	不可结算
冻结户 B	已发货未签收	不可结算
可结算户	已签收 + 过退货期	可发起提现

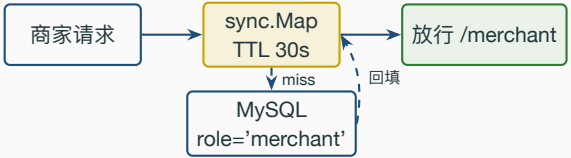
为什么必须拆账户：用户已支付未发货时退款 → 钱直接退回用户卡，**不能从商家已提现的钱里追**。”钱不出账户体系”之前商家无法 cash-out；一旦走到第三方银行追回成本极高（走法务）。

> 结算 = 把”什么时候可以给商家钱”，从拍脑袋变成**状态机**。

docs/slides/04-outbox-saga.tex 事务 + 事件原子写；*15-order-close-business.tex* 关单时机

admin vs merchant：边界由谁守

admin 多一个动作：promote 到 merchant + 30s 缓存



- **现状**：`PromoteToAdmin` 硬编码 RoleAdmin。**路线**：抽象成 `PromoteToRole(target, role)`。
- **流程**：BD 谈妥 → 商家传资质 → admin 审核 → `promote(role="merchant")` → 解锁 `/merchant`。
- **对称性陷阱**（deck 14）：提权慢可接受、降权慢是真风险。封号必须显式 `InvalidateRoleCache`，否则违规商家还能操作 30s。

service/admin.go:47–62 PromoteToAdmin 当前写死；middleware/rbac.go:22–45,88–91 cache + Invalidate

merchant 不能跨店 = 后端必须强约束

- **前端做不到**：哪怕 UI 完全不展示别店数据，商家可以**改 URL 参数 / 改 boss_id 字段**发请求。
- **鉴权做不到**：`RequireRole("merchant")` 只能保证”是某个商家”，不能保证”是这单的商家”。
- **必须在 dao / service 层强约束**：所有 merchant 接口的 SQL **必须**带 `WHERE boss_id = ctx.userID`。
- 类比 deck 14 的安全等级：
  - L1 路由前缀（区分 user/merchant/admin）——RBAC 中间件。
  - L2 业务约束（boss_id 必须匹配 ctx）——写在 service / dao。
  - L3 数据审计（异常聚合查询）——待补。

repository/db/dao/product.go:67–72 已有 WHERE id AND boss_id 模板可参考

压测数据告诉我们什么

商家维度查询的性能基线

- `order` 表当前 ~6M 行，`product` 表 ~956K 行（stressTest/REPORT.md L8）。
- C 端 `/orders/list`（WHERE user_id 走索引）：**58,406 RPS / p95 5ms**。
- 旧版深分页（无游标）：**8.3 RPS / p95 15.95s** ——大表上不走索引就是这个结局。
- **商家查"我今天的订单"** 如果不建 `idx_boss_date(boss_id, created_at)` → 直接对标深分页那一行。

> 先建索引，再开接口。否则上线第一天就被自家商家 DoS。

stressTest/REPORT.md 各链路压测表（行 3 vs 行 9）

遗留风险与路线图

当前的洞 + 修补顺序 + 边界由谁定

洞：

- **隐式商家**：任何用户调一次 product/create → 自己变商家。
- **无商家接口**：自助看数据走客服或 admin。
- **无 boss_id 索引**：查询一上来全表扫。
- **无资质 / 结算表**：法务 / 财务都不闭环。

修补顺序（按 ROI）：

1. RoleMerchant + RequireRole → 锁 product/create。
2. idx_boss_date 索引 + 三个读接口。
3. 库存告警 outbox + 钉钉机器人。
4. 资质审核表 + admin 审核流程。

边界由谁定：

决定	由谁负责
谁能成为商家	BD + 法务
商家可见字段	产品 + 风控
库存告警阈值	商品 + 数据
结算冻结期	财务 + 风控
提现单日限额	财务 + 风控
封禁商家触发	运营 + 法务

Q&A（上）：产品 / BD 角度

Q1：为什么不直接给商家开 admin 权限？

A：admin 能看全平台数据，商家拿到 admin = 当场翻车（看到别家销售）。RBAC 三层墙的意义就是**最小权限**。

Q2：merchant 看自店订单万一查询慢怎么办？

A：同 deck 13 思路——先建 `idx_boss_date` 索引、再上游标分页（last_id）；首页加 5min Redis 缓存兜底。C 端 orders/list 跑到 58K RPS，商家版数据量更小，吞吐预期类似。

Q3：商家” 品牌主页” 和” 店铺主页” 是不是一回事？

A：业务上不是。品牌主页是 C 端引流入口（公开 GET，匿名可见），店铺主页是商家自助后台（authed + RequireRole）。数据源可同源，但路由 / 鉴权 / 缓存策略完全不同。

Q&A（下）：法务 / 风控角度

Q4：第三方店铺和自营商品怎么区分？

A：自营 = BossID 指向”平台账号”（一个特殊 user_id）；第三方 = BossID 指向真实商家。聚合查询时一条 WHERE 就能区分。这也是为什么必须保留 BossID 而不是把自营 boss_id 写成 0。

Q5：资质审核怎么落地？OCR 还是人工？

A：OCR 做**字段抽取**（营业执照号 / 法人 / 经营范围），但**最终拍板必须人工**——营业执照能 PS、品牌授权能伪造。OCR 是节省人工录入，不是替代审核。落地：商家上传 → OCR 预填 → admin 审核员复核 → promote 到 merchant。

Q6：一个店铺多个员工账号怎么权限拆？

A：MVP 阶段不拆，单店铺 = 单账号。等业务量足够大再加 `merchant_employee` 表 + 子角色（运营 / 客服 / 财务）。当前不要预先抽象——角色越多越容易出”谁能批准谁”的扯皮（deck 14 同样论点）。

代码位置一览

BossID @ order *repository/db/model/order.go:12*

BossID @ product *repository/db/model/product.go:23*

BossID @ cart *repository/db/model/cart.go:10*

BossID @ favorite *repository/db/model/favorite.go:11–12*

BossId @ skill_product *repository/db/model/skill_product.go:6*

现有 admin group（merchant 模板） *routes/router.go:135–141*

product/create 当前位置 *routes/router.go:60*

RequireRole 中间件 *middleware/rbac.go:48–86*

InvalidateRoleCache *middleware/rbac.go:88–91*

PromoteToAdmin（待泛化） *service/admin.go:47–62*

outbox publisher（告警通道） *service/outbox/publisher.go:65–86*

merchant group 待新增 *routes/router.go:135 后插入新 group*

配套压测数据：stressTest/REPORT.md 链路压测表（C 端 orders/list 行 vs 深分页行）。